Mahavir Education Trust's

Shah & Anchor Kutchhi Engineering College

An Autonomous Institute Affiliated to University of Mumbai

UG Program in Information Technology

Report	
Course Name:	CLOUD COMPUTING
Title :	TaskFlow Cloud: A Cloud Based Student Task and Study Management System
Group Members Details	
Roll Number	Name of Student
124BTIT1039	KRISH BHANUSHALI
124BTIT1070	HET FINAVIA

TABLE OF CONTENTS

Description	Page No.
1. Case Selection	1 - 2
2. Problem Identification and Understanding	3 - 4
3. Need for Solution	5 - 7
4. Design Methodology	8 - 12
5. Implementation	13 - 19
6. Analytical Validation	20 - 23
7. Evaluation Trade offs and Decision Justification	24 - 27
8. Conclusion	28 - 29
9. References	30
10. Certificate	31

1. Case Selection

Selected Case

TaskFlow Cloud: a cloud-ready student task and study management system for planning academic work, tracking progress, and keeping study documents in one secure workspace.

Justification for Case Selection

Students commonly manage assignments, deadlines and notes through paper planners, informal messages and separate storage locations. These methods make it easy to miss deadlines, lose files and lose sight of overall progress. TaskFlow Cloud is a relevant cloud-computing case because it combines a responsive web interface, authenticated application services, structured data and managed file storage.

Relevance to Cloud Computing

- A browser-based service can be accessed from any device with an internet connection.

- The cloud design separates frontend hosting, API processing, relational data and local file storage.

- AWS Amplify, Amazon EC2 and Amazon RDS for PostgreSQL provide managed building blocks for deployment.

- The local development version uses the same application code with SQLite and local storage, allowing safe testing before cloud deployment.

Objectives of the Project

- Provide secure registration, login, logout and profile update features.

- Allow students to create, view, edit, complete and delete personal tasks.

- Support subject, priority, due date, search, filtering and sorting of tasks.

- Show dashboard analytics derived from the signed-in student’s own records.

- Allow validated upload, download and deletion of study files.

- Ensure one user cannot access another user’s tasks or files.

- Provide a cloud deployment design that uses Amplify, EC2 and RDS.

- Document tests, limitations, security controls and future enhancement opportunities.

Scope of the Project: The implemented application includes a React frontend, Flask REST API, JWT authentication, task management, dashboard analytics, study-file handling and local storage for development. AWS deployment files and instructions are provided, but no live AWS deployment is claimed in this report.

Organisation of the Report: Sections 2 and 3 explain the problem and requirements. Sections 4 and 5 present the design and implementation. Sections 6 and 7 validate and evaluate the solution. The remaining sections give the conclusion, references and certificate.

2. Problem Identification and Understanding

Students manage several courses at once, each with assignments, examinations, notes and deadlines. Without a central planning system, tasks are scattered across notebooks, chats and personal reminders. The result can be missed deadlines, duplicated effort and difficulty assessing progress.

2.1 Existing System

- Tasks are written in notebooks or remembered informally.
- Deadlines may be stored in different calendar, chat or note applications.
- Study material is kept in folders without a connection to the related task.
- Progress is difficult to measure across subjects.
- There is no reliable per-user access boundary when shared devices or records are used.

2.2 Problem Statement

The problem is to provide students with one secure, easy-to-use system for planning and tracking academic work. The system must retain data reliably, show useful progress information, protect each student's data, and provide an approach that can move from local development to a cloud deployment.

2.3 Comparison of the Existing Process and TaskFlow Cloud

Aspect	Manual or Scattered Process	TaskFlow Cloud
Task recording	Notebooks, messages or memory	Structured records with title, due date, subject, priority and status
Progress visibility	Manual review of many sources	Dashboard totals, completion percentage and status chart
Study material	Separate folders or shared links	Validated study-file upload linked to the user and optionally a task
Privacy	Depends on manual separation	JWT authentication and ownership-scoped API queries
Availability	Depends on physical notes or device	Cloud-ready browser application

2.4 Research Gap Addressed

General task applications may be too broad for a student project, while manual systems have no structured progress view or secure integration with study documents. TaskFlow Cloud addresses this gap with a focused student workflow and a transparent, deployable cloud architecture.

3. Need for Solution

A centralised digital system is needed to reduce missed deadlines, make current workload visible and keep study resources available with the relevant academic context. The solution needs a responsive interface, secure identity handling, reliable persistence and an upgrade path to managed cloud services.

Key Requirements of the Solution

- Centralised task records per student.
- Secure sign-in with a protected session.
- Task creation, editing, status changes and deletion.
- Search, filters, sorting and overdue highlighting.
- Dashboard statistics calculated from real user data.
- Study-file validation and protected access.
- A local development setup and documented AWS deployment architecture.

3.1 Technology Review

React and Vite provide a component-based, fast frontend. Flask provides lightweight REST endpoints in Python. SQLAlchemy supports SQLite in development and PostgreSQL in the cloud through a configuration change. JWT is used for authenticated API access, and bcrypt protects password hashes. For cloud deployment, AWS Amplify can host the frontend, EC2 can run Flask through Gunicorn and Nginx, RDS can provide PostgreSQL, and the EC2 host can retain uploaded study files on local disk.

3.2 Glossary of Technical Terms

Term	Meaning
API	Application Programming Interface used for frontend-backend communication.
JWT	JSON Web Token used to identify an authenticated user for a limited period.
React	JavaScript library used to build the browser interface.
Flask	Python web framework used for the REST API.
RDS	Amazon Relational Database Service; planned managed PostgreSQL database.
Local storage	Directory on the application host used for uploaded study-file bytes.
CORS	Browser security rule that requires the API to allow the frontend origin.
ORM	Object Relational Mapper; SQLAlchemy maps application models to database tables.
HTTPS	Encrypted HTTP transport used for a production deployment.

3.3 Functional and Non Functional Requirements

Category	Requirement
Functional	Register, login, logout and update profile.
Functional	Create, edit, list, filter, search, sort, complete and delete own tasks.
Functional	View dashboard totals, completion percentage, upcoming and recent tasks.
Functional	Upload, download and delete owned PDF, TXT and DOCX study files up to 5 MB.
Security	Hash passwords with bcrypt; use JWT; enforce ownership checks on every private endpoint.
Usability	Use responsive layouts, validation messages, loading states, empty states and deletion confirmation.
Maintainability	Separate frontend components, API routes, models, local storage and deployment configuration.
Cloud readiness	Use the same data model locally and in PostgreSQL, with application-host local storage.

4. Design Methodology

TaskFlow Cloud follows a three-tier design. The browser presents the interface and sends REST/JSON requests. The Flask API applies validation and business rules. The database persists structured records and the storage adapter handles study-file bytes. In cloud mode, each tier maps to an appropriate AWS service.

4.1 Design Principles

- Separation of concerns: user interface, API logic, database models and storage are independent modules.

- Least privilege and data isolation: the user identity comes from the JWT, never from a client-supplied user ID.

- Validation at multiple layers: forms provide quick feedback and the API remains authoritative.

- Cloud portability: configuration selects SQLite for development or PostgreSQL for deployment while retaining local files on the application host.

- Honest verification: only locally tested functions are reported as verified; AWS deployment remains planned.

4.2 System Components

Component	Technology	Responsibility
Frontend	React, Vite, Axios, Recharts	Pages, navigation, forms, task presentation, dashboard charts and API calls.
Authentication	Flask JWT Extended, bcrypt	Registration, login, token verification and profile access.
Task service	Flask routes and SQLAlchemy	Task CRUD, filters, sorting, pagination and status updates.
Dashboard	Flask dashboard route	Calculates totals, overdue count, completion percentage, upcoming and recent tasks.
File service	Local storage adapter	Validates files and stores bytes on the application host.
Database	SQLite locally; PostgreSQL on RDS	Users, tasks and study-file metadata.
Cloud hosting	Amplify, EC2, RDS	Planned production deployment topology.

4.3 Proposed System Architecture

- A student opens the React single-page application in a browser.

- The student registers or logs in; the API returns a time-limited JWT.

- Axios sends the token in the Authorization Bearer header for private requests.

- Flask validates the request and uses the authenticated identity to scope queries.

- SQLAlchemy reads or writes task and metadata records.

- The local storage adapter stores study-file bytes on the application host.

- The API returns JSON or an authenticated file response and the frontend updates the view.

{screeshots}

Figure 4.1: TaskFlow Cloud system architecture

4.4 Data Model

Entity	Key fields	Relationship
users	id, name, email, password_hash, timestamps	One user has many tasks and study files.
tasks	id, user_id, title, description, subject, priority, status, due_date, timestamps	Each task belongs to exactly one user.
study_files	id, user_id, task_id, original_filename, storage_key, content_type, file_size, uploaded_at	Each file belongs to one user and may optionally reference a task.

4.5 API Design

Area	Representative endpoints	Purpose
Health	GET /api/health	Reports backend and database status.
Authentication	POST /api/auth/register; POST /api/auth/login; GET/PATCH /api/auth/me	Account lifecycle and profile management.
Tasks	GET/POST /api/tasks; GET/PUT/DELETE /api/tasks/<id>; PATCH status	Task lifecycle, search, filters, sorting and ownership enforcement.
Dashboard	GET /api/dashboard/stats	Returns user-specific statistics and task summaries.
Files	GET /api/files; POST upload; GET download; DELETE file	Private study-file lifecycle.

5. Implementation

5.1 Frontend Implementation

The frontend is built with React and Vite. It uses page components for login, registration, dashboard, task lists, task details, task forms, notes and profile management. Reusable components provide navigation, task cards, filters, priority and status badges, loading indicators, empty states, error messages and deletion confirmation. Axios service modules centralise API calls and attach the JWT token.

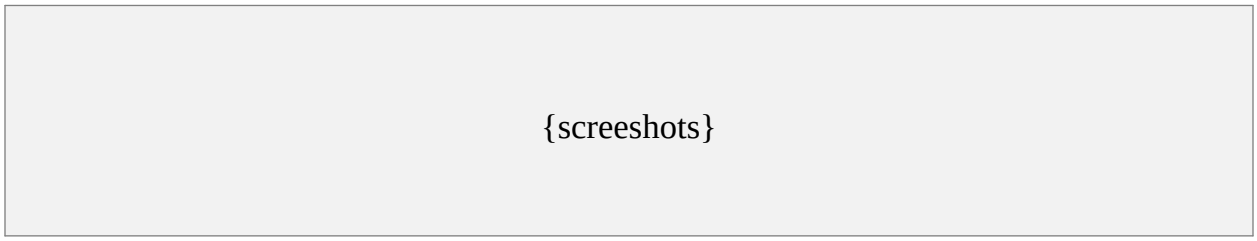

Figure 5.1: Login or registration screen

5.2 Authentication and User Isolation

The backend stores bcrypt password hashes, never plaintext passwords. On successful login or registration, it returns a JWT. Protected routes derive the current user from that token. Task and file queries always include the authenticated user ID, and an absent or foreign record produces a not-found response rather than disclosing another user’s data.

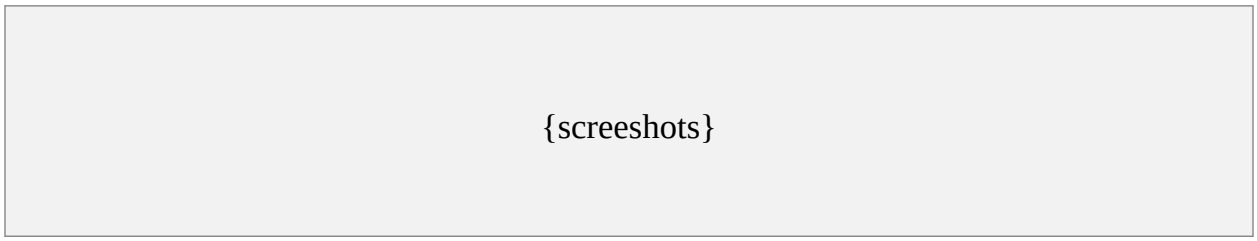

Figure 5.2: Authenticated dashboard

5.3 Task Management

Students can create tasks with a title, optional description and subject, priority, status and due date. The task list supports text search, status, priority and subject filters, sorting, pagination at the API level and overdue highlighting. Status values are Pending, In Progress and Completed. A confirmation dialog appears before deletion.

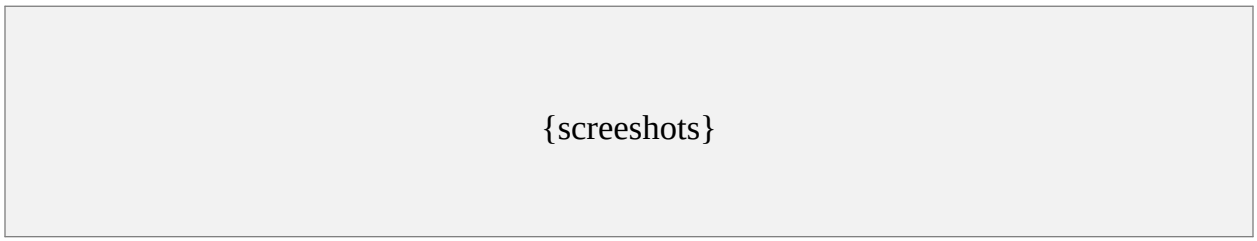

Figure 5.3: Task list with filters and overdue highlighting

5.4 Dashboard Analytics

The dashboard calls GET /api/dashboard/stats and displays totals, pending, in-progress, completed and overdue counts, completion percentage, upcoming deadlines, recent tasks and status charts. Values are calculated server-side from the signed-in user’s records; they are not hardcoded frontend examples.

{screeshots}

Figure 5.4: Dashboard statistics and task status chart

5.5 Study File Management

The notes area accepts PDF, TXT and DOCX files. Client and server validation reject unsupported types and files over the configured 5 MB limit. File bytes are stored on the application host's local disk while the database stores metadata and a storage key. Downloads are authorised and served through the API.

{screeshots}

Figure 5.5: Study notes upload and file list

5.6 Local and Cloud Deployment Configuration

During local development, the Vite frontend runs on port 5173 and the Flask API runs on port 5000. SQLite and local disk storage remove the need for cloud resources during testing. The documented production configuration uses Amplify for the static frontend, an EC2 instance running Gunicorn behind Nginx for the API, private RDS PostgreSQL for relational data, and local disk storage on the EC2 host for study-file bytes. Environment variables keep credentials out of source code.

{screeshots}

Figure 5.6: Backend health endpoint or deployment configuration

6. Analytical Validation

Validation was completed through automated backend tests, a production frontend build and a manual end-to-end test script. The AWS architecture is documented and configured in project files, but it has not been deployed from this workspace. Therefore, cloud-runtime behaviour is not marked as verified.

6.1 Validation Approach

- Run backend pytest tests for authentication, tasks, ownership, dashboard, files and error shapes.

- Run npm run build to verify that the frontend production bundle compiles.

- Perform a manual flow: register, create tasks, observe dashboard updates, upload and download a file, delete a task, update profile and log out.

- Review configuration and deployment documents for AWS service boundaries and security controls.

6.2 Test Case Analysis

ID	Test	Expected Result	Observed Result	Status
TC-01	Register and login	Authenticated user and token returned	Automated tests passed	Passed
TC-02	Private route without token	API rejects request	Auth guard test passed	Passed
TC-03	Create and edit task	Task persists with valid fields	Task tests passed	Passed
TC-04	Foreign task access	Other user cannot view or modify task	Returns 404; ownership test passed	Passed
TC-05	Dashboard statistics	Counts reflect own task data	Dashboard stats test passed	Passed
TC-06	Invalid task input	Readable validation error returned	Validation test passed	Passed
TC-07	File type and size checks	Invalid uploads rejected	File validation tests passed	Passed
TC-08	File download and delete	Only owner can download/delete	Ownership and lifecycle tests passed	Passed
TC-09	Frontend production build	Vite bundle compiles	npm run build passed	Passed
TC-10	Local end-to-end flow	Core student workflow succeeds	Manual script documented and completed	Passed
TC-11	Live AWS deployment	Cloud services run and integrate	Not deployed from this workspace	Pending

6.3 Results Summary

The backend test suite reports 11 passing tests. These cover registration, duplicate-email handling, invalid credentials, protected routes, task lifecycle operations, user isolation, validation, dashboard statistics, file validation, file ownership and consistent JSON errors. The frontend production build also passes. The resulting evidence supports the implemented local application. A live AWS test remains pending because no AWS account resources were created or operated for this report.

6.4 Limitations Identified During Validation

- JWTs are stored in localStorage; HttpOnly cookie sessions would reduce exposure to XSS.

- There is no password reset or email verification flow.

- The API supports pagination, but the frontend does not yet present pagination controls.

- The upload limit is a single 5 MB limit for all permitted file types.

- A live AWS deployment, performance test and cloud monitoring verification have not been performed.

7. Evaluation Trade offs and Decision Justification

7.1 React and Vite Frontend

React was selected for reusable interface components and state-driven rendering. Vite gives a simple local development and production-build workflow. The trade-off is a separate JavaScript build pipeline, but the component structure makes the dashboard, task views and protected routes easier to maintain than a server-rendered prototype.

7.2 Flask API on EC2

Flask was selected because it is lightweight, clear for a student project and integrates naturally with SQLAlchemy and Python validation code. EC2 is the documented cloud target because it makes the web-process model visible: Gunicorn runs the application, systemd supervises it and Nginx provides reverse proxy and HTTPS handling. The trade-off is that EC2 requires operating-system maintenance, unlike serverless compute.

7.3 RDS PostgreSQL

SQLite is ideal for zero-cost local development, while PostgreSQL on RDS is suitable for managed production relational storage. The project uses relational associations, uniqueness and indexes that fit SQL well. RDS reduces database administration but introduces ongoing charges and VPC/security-group configuration.

7.4 Local File Storage

Local file storage was selected for uploaded study material because it keeps the deployment simple and avoids a separate object-storage service. The database retains metadata and a storage key, while bytes remain on the application host. The trade-off is that the EC2 host requires disciplined backup, disk-capacity monitoring and a future review if file volume grows.

7.5 Overall Trade off Summary

Decision	Chosen Service or Tool	Alternative	Reason
Frontend hosting	AWS Amplify	Self-managed static server	Managed HTTPS hosting and simple SPA deployment path.
API compute	EC2 with Gunicorn and Nginx	AWS Lambda	Straightforward process model for a Flask student project.
Relational data	RDS PostgreSQL	Self-managed database VM	Managed database and same SQLAlchemy model as local mode.
File bytes	Local disk on EC2	Store bytes in database	Simple implementation with metadata held separately from bytes.
Development database	SQLite	Always use cloud PostgreSQL	Fast local setup with no paid infrastructure.

7.6 Security Considerations

Area	Control or Recommendation	Status
Passwords	bcrypt hashes; no password hash in API responses	Implemented
Authentication	JWT required for private routes; token expiry configured	Implemented
Ownership	User identity derived from JWT; foreign objects are not revealed	Implemented and tested
Uploads	Extension, MIME and file-size validation	Implemented and tested
Host storage	Restrict filesystem permissions and maintain backups of uploaded files	Documented for deployment
Network	HTTPS, private RDS, least-privilege security groups	Documented for deployment
Session hardening	Move from localStorage token to HttpOnly cookie	Future enhancement

7.7 Cost and Scalability Analysis

Local mode has no mandatory AWS charge. In cloud mode, Amplify, EC2 and RDS introduce usage-based or instance-based costs. The deployment guide recommends Free-Tier sizes where available, budget alarms and explicit cleanup of Amplify, EC2 and RDS snapshots. The application can scale further by placing the API behind an application load balancer, serving the frontend through CloudFront, adding database backups and alarms, and reviewing file-storage capacity as usage grows.

8. Conclusion

TaskFlow Cloud is a completed local full-stack application for student task and study management. It provides authenticated accounts, private per-user task management, dashboard analytics and validated study-file handling. The implementation cleanly separates the React frontend, Flask API, relational data model and storage layer.

The project has been validated through automated backend tests, a successful frontend production build and a documented manual workflow. These results support the claim that the local application functions as designed. The AWS deployment topology is intentionally described as cloud-ready rather than deployed: no paid cloud infrastructure was created or tested for this report.

Objective Status Summary

No.	Objective	Status
1	Secure per-student registration, login and profile access	Verified
2	Full task lifecycle with valid input and deletion confirmation	Verified
3	Search, filters, sorting and overdue visibility	Verified
4	Dashboard analytics from current user data	Verified
5	Validated study-file upload, download and deletion	Verified
6	User data isolation through ownership checks	Verified
7	Cloud deployment architecture and setup documentation	Designed and documented; live deployment pending
8	Tests and transparent limitations	Verified

Key Learning Outcomes

- Understanding full-stack communication through REST/JSON APIs.
- Applying JWT authentication, bcrypt hashing and ownership-based authorisation.
- Designing data models that work locally with SQLite and in the cloud with PostgreSQL.
- Separating file metadata from file bytes through a storage adapter.
- Evaluating AWS services with cost, security, maintainability and operational trade-offs.
- Reporting verified outcomes separately from planned cloud deployment work.

Future Enhancements

- Deploy and test the documented AWS environment with a budget alarm.
- Add password reset, email verification and stronger session handling.
- Add task sharing, reminders and notifications.

- Add pagination controls, subject management and full-text search.
- Provide in-browser file previews and richer accessibility review.
- Add CI/CD, monitoring, host-storage backups and audit logs.
- Use CloudFront and an application load balancer for a more scalable production architecture.

9. References

1. React Documentation. https://react.dev/
2. Vite Documentation. https://vite.dev/guide/
3. Flask Documentation. https://flask.palletsprojects.com/
4. Flask JWT Extended Documentation. https://flask-jwt-extended.readthedocs.io/
5. SQLAlchemy Documentation. https://docs.sqlalchemy.org/
6. AWS Amplify Hosting Documentation. https://docs.aws.amazon.com/amplify/
7. Amazon EC2 User Guide. https://docs.aws.amazon.com/ec2/
8. Amazon RDS for PostgreSQL User Guide. https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/
9. OWASP Authentication Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
10. MDN Web Docs: HTTP Access Control. https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CORS

10. Certificate

Project completion certificate or relevant cloud training certificate may be inserted here.

{screeshots}

Figure 10.1: Certificate placeholder